Andrea Colombo - Algorithms for Massive Datasets course project - A.A 2025/2026

Andrea Colombo

Abstract — This report contains the documentation related to the project submission for the *Algorithms for Massive Datasets* course. Taught by *Dario Malchiodi* at *Università degli studi di Milano.*
We present the implementation of two stream analysis algorithms: Flajolet-Martin and Alon-Matias-Szegedy, starting from the theory behind them and going all the way through the implementation choices and experimental results.

Table of Contents

1 Introduction

The analysis of massive datasets has become an increasingly important problem in the later years. These scenarios pose significant challenges due to the sheer amount of data to be proceseed; in most cases, storing the entire dataset in memory is not feasible.
Stream analysis algorithms address this issue by processing the dataset sequentially, using limited amounts of memory and providing estimations about the stream properties we are interested in. Since these algorithms do not need to store the entirety of the stream in memory, they are used across a moltitude of domains and hardware, ranging from huge server rigs to tiny embedded microcontrollers.

The algorithms and tasks we are going to explore are the following:

- Using the *Flajolet-Martin Algorithm* [1] to produce and estimation of the unique userIDs present in the dataset. The full description of the algorithm and the implementation choices can be found in Section 4.

1.1 Development Environment

This project was developed using *Google Colab* as the main computation environment. The Jupyter Notebook submitted alongside this project may require some modifications before being run in a local environment.
Below is a brief description of the computational resources available on the CoLab runtime and the versions of the libraries used in this project:

- **CPU**: Intel Xeon CPU with 2 vCPUs (virtual CPUs) and 13GB of RAM.
- **Python Version**: 3.13.15 (built with GCC 11.4.0)
- **PySpark Version**: 4.0.4
- **xxHash Version**: 4.0.1

> Please note that these versions are correct at the time of writing: 2026-09-01.

2 Dataset Description

The dataset used for this project is the **New York Times Articles & Comments (2020)** [2] dataset, freely available and licensed under the **CC-BY-NC-SA-4.0** license.

The datset, once downloaded, has a size of approximately **6.15 GBs** presents itself in the form of multiple files:

- **nyt-articles-2020.csv**: Contains all the articles published in 2020 by the NYT.
- **nyt-comments-2020.csv**: this file contains all the comments relative to the articles found in *nyt-articles-2020.csv*.
- **nyt-comments-part0.csv .. nyt-comments-part9.csv**: these fails simply contain the data found in *nyt-comments-2020.csv* split in 10 different partitions.
- **test.csv**: Not relevant for our use case

- **train.csv**: Not relevant for our use case

2.1 Preprocessing Techniques

During the preprocessing phase of the project we discarded all the columns that are not relevant for our use case, in particular:

- For the Flajolet-Martin portion of the project (Section 4), we discarded all the columns except the *UserID* and, since all the fields inside the schema are tagged as **nullable**, we excluded all the null entries to avoid using garbage data.

2.2 Subsampling

In order to ensure a reasonable execution time, we introduced a method that allows the user to load only a part of the dataset instead of the whole. This method can be tweaked by modifying the **SAMPLING_PROPORTION** (see: Section 3) variable inside the notebook provided alongside this document.

3 System Configuration

The python notebook submitted with this project can be configured with the following set of variables:

- **ENABLE_LOGGING**: enables additional prints during the notebook execution
- **SAMPLING_PROPORTION**: How much of the whole dataset we want to use for the current run, valid if in range $(0, 1]$.
- **FM_NUM_HASHES**: Number of hash functions to use for the Flajolet-Martin implementation.

4 Flajolet–Martin Algorithm

First introduced in 1985 [1], the Flajolet-Martin algorithm is a probabilistic algorithm that aims at estimating the number of distinct elements through the use of hash functions.
The core idea is to leverage two very important properties of hash functions:

- **Determinism**: the hash function always maps the same value to the same result
- **Uniform Distribution**: the hashed values are uniformly distributed over the binary space

But why do we focus on the number of trailing zeros (tail length) of the hashed value?
The probability that a single hash value ends with n trailing zeros is $\frac{1}{2^n}$.
From this forumula we can derive that:

- The probability of a single hash value not having n trailing zeros is $1 - \frac{1}{2^n} = 1 - 2^{-n}$
- The probability of **none** of k hash values having n trailing zeros is $\left(1 - \frac{1}{2^n}\right)^k$. This can be approximated to $e^{-k2^{-n}}$ for very large numbers.

Therefore, if the maximum number of trailing zeros is $R_{\max}$, it implies that we have seen approximatively $2^{R_{\max}}$ unique elements so far inside the stream.

4.1 The Outlier Problem

Using a single hash function has one big issue, since we are taking the maximum number of trailing zeros from a single source, we are susceptible to outliers.

The solution is simply using multiple hash functions and compute the median of their results. This has been verified to provide more accurate results but it still has one problem, it always results in estimates that are power of 2.
In order to fix this issue, we compute the average of the median estimations we just computed.

4.2 Space/Time Complexity

This algorithm has a time complexity of $O(kn)$ where n is the length of the stream and k is the constant value that refers to the computational cost of the hash functions and update procedure. The space complexity is $O(k)$ instead, we only need to store a constant amount of information needed to update the trailing zeros counts; the precise memory usage depends on how many hash function we decide to use for the algorithm.

4.3 Implementation Details

4.3.1 Hash Function Choice

The hash function used in the submitted implementation is xxhash, an extremely fast non cryptographic hashing algorithm. In particular, this implementation uses the xxh3 64 bits version, xxhash is one of the most used hashing algorithms for non cryptographic uses in real world use cases (PySpark uses it too).

5 Algorithm implementation

python

```python
def ctz(x):
    if x == 0:
        return 64
    return (x & -x).bit_length() - 1

def hash64bits(value, seed):
    return xxhash.xxh64(value.encode("utf-8"), seed=seed).intdigest()

def fm_partition(it):
    registers = [0] * FM_NUM_HASHES
    for user in it:
        for i in range(FM_NUM_HASHES):
            bits = hash64bits(user, i)
            r = ctz(bits)
            if r > registers[i]:
                registers[i] = r
```

```
    yield registers
```

5.1 Experimental Results

One of the most important questions ask ourselves when implementing this algorithm is the following:

> How many different hash functions do we use?

This is a very important questions because it requires us to think about the tradeoffs between compotational expense and accuracy for our specific use case.
Choosing to use more hash functions will result in more accurate estimations at the expense of performance.

6 AMS Algorithm

6.1 Space/Time Complexity

6.2 Implementation Details

6.3 Experimental Results

7 Conclusion

7.1 Future Work

8 Plagiarism and AI Usage Statement

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study. No generative AI tool has been used to write the code or the report content.

Bibliography

[1] F. Philippe and M. G. Nigel, "Proba bit istic Cou nting Algorithms for Data Base Applications," *Journal of computer and system sciences 31*, 1985.

[2] B. Dornel, "New York Times Articles & Comments (2020)." 2021.